

WeKan L1 Round Debrief & Fix Pack

Every question that was actually asked — with the answer you should have given

Satish Kumar · SDE-3 Node.js · August 2026

0. What Actually Happened

The preparation went deep — V8, libuv, the six event-loop phases, streams, backpressure, WiredTiger cache. The round went **wide and shallow**. That mismatch is the whole story of this interview, and it is worth naming precisely, because the fix is not "study more".

The real failure mode You were not asked anything you don't know. You were asked **"What is Express?"** and could not retrieve a sentence you have known for years. That is a **retrieval failure under load**, not a knowledge gap. Deep material crowds the working memory; simple definitions get pushed out precisely when they are asked for.

The questions, as asked

#	Question	Outcome	Fixed in
1	Coding: <code>minMeetingRooms(intervals)</code> in JavaScript, without mutating the input	8 of 18 tests failed	Section 1
2	Most challenging project you have worked on	Needs structure	Section 7
3	Have you worked on CI/CD? Now explain it using GitHub Actions	Needs specifics	Section 3
4	You've used Jest — name a function that exists in it	Trivia recall	Section 4
5	MongoDB aggregation: average salary per department	Correct	Section 5
6	Implement an LRU cache instead of using Redis	Needs code fluency	Section 6
7	What is Node.js? What is React.js? What is Express.js?	Blanked	Section 2
8	Explain your AI/RAG project — and name the exact LLM model	Model name vague	Section 7

What this tells you about WeKan-style rounds The bar was **breadth of everyday practice**, not depth of runtime internals. Coding correctness, a testing tool you can name functions from, a working CI pipeline, a cache you can write from scratch, and definitions you can produce in one second. Nothing in this list needed the event loop diagram. Keep that material — it wins L2 and architecture rounds — but never again enter an L1 without the boring layer rehearsed.

How to use this document

- Section 2 first.** Memorise those five definitions word for word. They cost ten minutes and they were the most expensive thing you lost.
- Section 1 second.** Re-solve the meeting-rooms problem from a blank file before reading the solutions.
- Sections 3–6 are recall material — read once, then self-test from Appendix A.
- Section 7 is narrative — rehearse out loud, not silently.

One piece of perspective Failing on definitions after preparing internals is frustrating, but it is a **cheap** failure to fix — a weekend, not a quarter. The expensive failure would have been the reverse: not knowing the internals at all. You have the hard half. This document adds the easy half.

1. The Coding Question — Minimum Meeting Rooms

The problem Given `intervals` where each entry is `[start, end]`, return the minimum number of rooms required. A meeting ending at time `t` frees the room for a meeting starting at `t`. **The original input array must not be modified.**

1.1 What you submitted, and exactly why it failed

```
// SUBMITTED - 10 passed, 8 failed
function minMeetingRooms(intervals) {
  if(intervals === null || intervals.length === 0) return 0;
  if(intervals.length === 1) return 1;
  intervals.sort((a,b) => (a[0]-a[b])); // BUG 1 and BUG 3

  let maxRooms = 1;
  for(let i=0; i<intervals.length; i++) {
    const end = intervals[i][1];
    let rooms = 0; // BUG 4
    for(let j=i+1; j<intervals.length; j++) {
      if(intervals[j][1] < end || intervals[j][0] < end) { // BUG 2
        rooms++;
      }
    }
    maxRooms = Math.max(maxRooms, rooms);
  }
  return maxRooms;
}
```

Bug 1 — the comparator typo

`(a,b) => a[0] - a[b]`. The second term should be `b[0]`. What you wrote is `a[b]`: `b` is an *array* being used as an object key, so `a[b]` is `undefined`, and `number - undefined` is `NaN`. Every comparison returns `NaN`.

Why this is dangerous rather than merely wrong A comparator returning `NaN` does not throw. V8 treats it as "no opinion" and leaves the order roughly as it was. So the sort **silently did nothing** and the failures showed up later, in the counting logic, where you would never think to look. A comparator that returns anything other than a number is one of the quietest bugs in JavaScript.

Bug 2 — the overlap condition is not an overlap condition

`intervals[j][1] < end || intervals[j][0] < end` asks "does meeting *j* start or finish before meeting *i* finishes?" That is not overlap. Two meetings `[1,4]` and `[2,5]` overlap; so do `[1,8]` and `[2,3]`. Correct overlap between `[s1,e1]` and `[s2,e2]`, given that touching endpoints do *not* conflict, is:

```
s1 < e2 && s2 < e1
```

Bug 3 — the input is mutated

`Array.prototype.sort()` sorts **in place**. It returns the same array reference, it does not make a copy. The harness explicitly checks `inputUnmodified`.

The trap you got lucky on Your `inputUnmodified` checks all passed — but only *because* the comparator was broken and nothing moved. Had you written `a[0] - b[0]` correctly, the sort would have reordered the array and you would have failed on mutation instead. Two bugs cancelling out. Verified: sorting `[[5,10],[1,5],[10,15]]` with a correct comparator returns `inputUnmodified: false`.

The habit: the moment a problem says "do not modify the input", copy first — `[...intervals]` for the outer array, or `intervals.map(i => [...i])` if you will touch the inner arrays too. Do it in the first line, before you think about the algorithm.

Bug 4 — the count never includes the meeting itself

`rooms` starts at 0 and the inner loop begins at `j = i+1`, so meeting *i* is never counted as occupying a room. Even with a correct overlap test the answer would be one short. The `let maxRooms = 1` initialisation papers over this for small inputs, which is why some tests still passed.

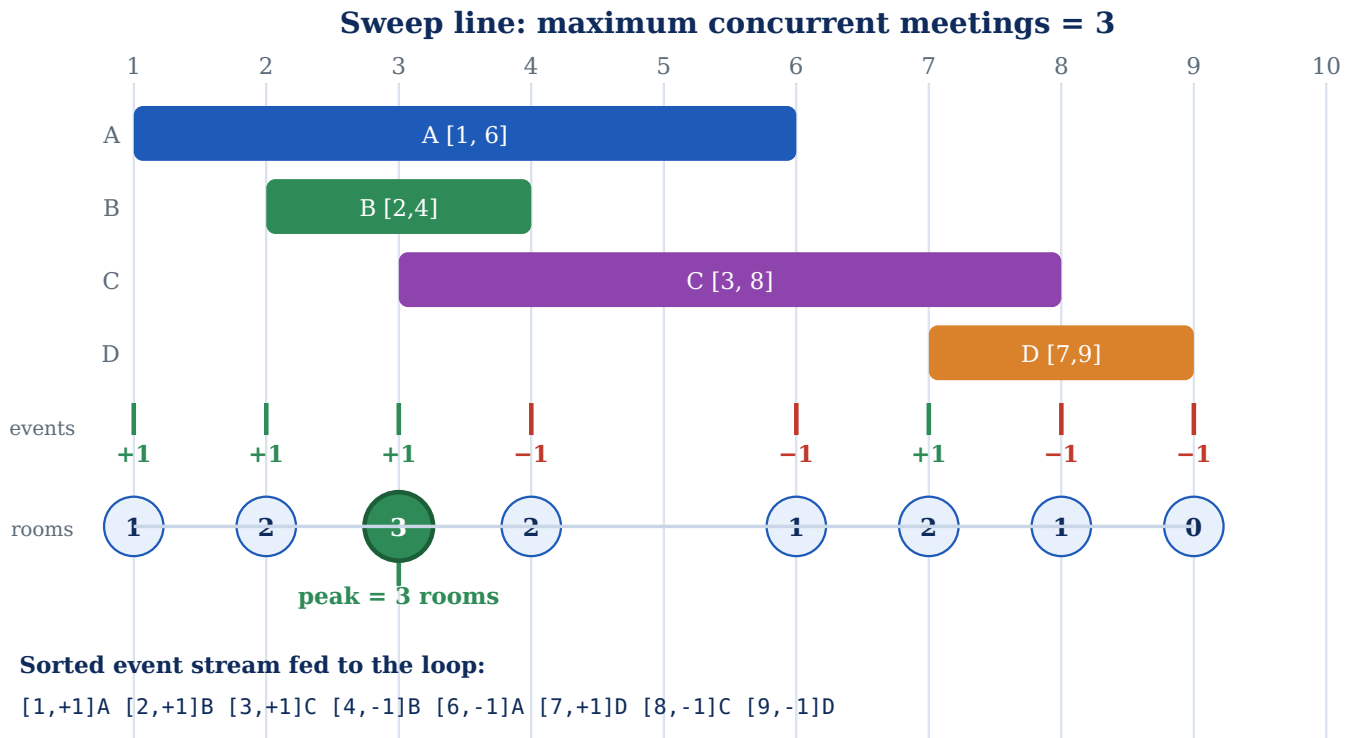
Bug 5 — the shape of the algorithm cannot work

The deeper issue: counting overlaps only *forward* from each meeting undercounts. In `[[1,4],[2,5],[3,6]]` all three overlap at time 3, so the answer is 3. Looking forward from `[1,4]` finds two later meetings, giving 2. Looking forward from `[2,5]` finds one, giving 1. You never see all three together. **The maximum concurrency at a moment in time is not visible from any single meeting's forward view.**

The reframe that unlocks the problem Stop thinking about meetings overlapping each other. Think about a **moment in time** and ask: how many meetings are running right now? The answer is the maximum of that count over all moments. And the count can only change at a start or an end — so you only need to check those instants, never the times in between.

1.2 The Sweep — the mental model behind every correct solution

Four meetings: **A** = [1,6], **B** = [2,4], **C** = [3,8], **D** = [7,9]. Walk left to right along the timeline. **+1** at every start, **-1** at every end. The answer is the highest the running counter ever reaches.



TIE RULE — the single detail that decides pass or fail

When a start and an end land on the SAME time, process the END first. [1,5] and [5,10] then need 1 room, not 2, because a meeting ending at t frees the room at t . In a sort comparator: tie-break by delta ascending (-1 before $+1$).

WHY PAIRWISE COUNTING CANNOT WORK

A, B and C are all live at time 3 — that is a three-way pile-up. Counting only forward from A sees 2, from B sees 1. No single meeting's view reveals the peak. You must count at a MOMENT, not per meeting pair.

1.3 Four working solutions, with trade-offs

All four were run against the full 18-test harness: **18 passed, 0 failed** for each, including the `inputUnmodified` checks.

Approach A — brute force by counting at each start time

The direct translation of "how many meetings are live at this moment". Only start times need checking, because the count can only rise at a start.

```
function minMeetingRooms(intervals) {
  if (!intervals || intervals.length === 0) return 0;

  let max = 0;
  for (const [t] of intervals) {          // t = a candidate moment
    let live = 0;
    for (const [s, e] of intervals) {
      if (s <= t && t < e) live++;        // started, not yet ended
    }
    max = Math.max(max, live);
  }
  return max;
}
```

Why it is safe No sorting at all, so the input is never touched — the mutation requirement is satisfied for free. The condition `s <= t && t < e` encodes the tie rule directly: a meeting ending exactly at `t` is not counted.

$O(n^2)$ time, $O(1)$ extra space. Perfectly acceptable as a first answer, and a genuinely good move under pressure: state that it is $O(n^2)$, say you will optimise next, and get a green test run on the board.

Approach B — two sorted arrays (recommended answer)

Separate the starts from the ends, sort each independently, then walk the starts while releasing any rooms whose end time has already passed.

```
function minMeetingRooms(intervals) {
  if (!intervals || intervals.length === 0) return 0;

  const starts = intervals.map(i => i[0]).sort((a, b) => a - b);
  const ends   = intervals.map(i => i[1]).sort((a, b) => a - b);

  let rooms = 0, max = 0, e = 0;

  for (let s = 0; s < starts.length; s++) {
    while (ends[e] <= starts[s]) { rooms--; e++; } // <= is the tie rule
    rooms++;
    max = Math.max(max, rooms);
  }
  return max;
}
```

The mutation problem solves itself `.map()` returns a **new** array. You are sorting those copies, never `intervals`. This is why B is the answer to give when the prompt forbids mutation — the copy is not a defensive afterthought, it falls out of the algorithm.

The pairing between a start and an end is deliberately thrown away. It does not matter *which* meeting freed a room, only that one was freed. **$O(n \log n)$ time, $O(n)$ space.**

Approach C — event sweep (the diagram, as code)

```
function minMeetingRooms(intervals) {
  if (!intervals || intervals.length === 0) return 0;

  const events = [];
  for (const [s, e] of intervals) {
    events.push([s, +1]);
    events.push([e, -1]);
  }
  // time ascending; on a tie, -1 (end) before +1 (start)
  events.sort((x, y) => x[0] - y[0] || x[1] - y[1]);

  let rooms = 0, max = 0;
  for (const [, delta] of events) {
    rooms += delta;
    max = Math.max(max, rooms);
  }
  return max;
}
```

The comparator idiom worth memorising `(x, y) => x[0] - y[0] || x[1] - y[1]`. When the primary keys are equal the first term is `0`, which is falsy, so `||` falls through to the tie-breaker. This is the standard JavaScript multi-key sort and it comes up constantly. Note also: `-1 - 1 = -2`, negative, so ends sort before starts. That is exactly the tie rule.

$O(n \log n)$ time, $O(n)$ space. Generalises further than B — if the question becomes "which time window is busiest" you already have the timestamps attached.

Approach D — min-heap of end times (the classic textbook answer)

The heap holds the end time of every currently occupied room. Its smallest element is the room that frees up soonest.

```
function minMeetingRooms(intervals) {
  if (!intervals || intervals.length === 0) return 0;

  const sorted = [...intervals].sort((a, b) => a[0] - b[0]); // copy, then sort
  const heap = new MinHeap();

  for (const [start, end] of sorted) {
    if (heap.size() && heap.peek() <= start) heap.pop(); // reuse a room
    heap.push(end);
  }
  return heap.size(); // rooms still occupied = rooms needed
}
```

The reason not to reach for this first in JavaScript JavaScript has **no built-in priority queue**. Java gives you `PriorityQueue`; Python gives you `heapq`. In a JS interview, choosing D commits you to hand-writing sift-up and sift-down — roughly 20 lines of index arithmetic `((i-1)>>1, 2*i+1, 2*i+2)` — under time pressure, for the same $O(n \log n)$ that Approach B gets in eight lines. **Know that D exists, name it, then write B.**

Only one subtlety: popping at most one element per meeting is correct, because each meeting adds exactly one room, so at most one can be freed for it.

Trade-off summary

Approach	Time	Space	Mutates input?	When to choose it
A Brute force	$O(n^2)$	$O(1)$	No — no sort	First pass when you want something correct on the board fast, or when n is small and clarity matters more
B Two sorted arrays	$O(n \log n)$	$O(n)$	No — <code>.map()</code> copies	The answer to give. Shortest correct optimal code, and it satisfies the no-mutation rule without extra work
C Event sweep	$O(n \log n)$	$O(n)$	No — builds a new array	When the follow-up asks <i>when</i> the peak occurs, or extends to weighted or multi-resource variants
D Min-heap	$O(n \log n)$	$O(n)$	No — if you spread-copy	Name it to show you know the canonical solution; write it only in a language with a built-in heap

1.4 How to run the next live-coding round

1. **Restate the tie rule before writing anything.** "A meeting ending at t frees the room at t — so `[1,5]` and `[5,10]` need one room." Say it out loud. It is the one rule that separates a passing solution from a nearly-passing one, and saying it proves you read the prompt.
2. **Copy the input in line one.** If the prompt mentions mutation at all, `const sorted = [...intervals]` before anything else.
3. **Brute force first, out loud.** "The $O(n^2)$ version counts live meetings at each start time. Let me get that working, then optimise." Interviewers reward this. A correct slow answer beats a fast wrong one every time.
4. **Dry-run one failing-shaped case before declaring done.** Three mutually overlapping intervals, `[[1,4],[2,5],[3,6]]`, would have caught this bug in thirty seconds. Pick a case where the answer is *more* than 2.
5. **Read your comparator character by character.** `a[0] - b[0]`. This single typo cost the question.

The 30-second spoken answer "The key insight is that you never compare meetings pairwise — you count how many are live at a given moment, and take the maximum. I'd sort the start times and the end times into two separate arrays, then walk the starts: before claiming a room for each start, I release every room whose end time is less than or equal to that start. The counter's peak is the answer. That's $O(n \log n)$, and because I sort copies produced by `.map()`, the original input is never touched. The textbook alternative is a min-heap of end times — same complexity, but JavaScript has no built-in priority queue, so the two-array version is less code and less risk."

2. The Basics That Got Missed

Read this first These are not concepts to understand — you already understand them. They are **sentences to memorise verbatim**, so that under pressure your mouth produces them while your brain is still catching up. Treat them like a phone number, not like a topic. Say each one aloud five times.

2.1 Library vs framework

Say exactly this "A library is code I call. A framework is code that calls me. That's inversion of control. With Axios, I decide when to invoke it and control comes straight back to me. With Spring Boot, I write a controller method and Spring decides when to invoke it — when a request arrives."

What "code" means here: somebody else's code — code you did not write, pulled in from npm or Maven. The real question the distinction answers is *whose functions call whose*. With a library, your function calls their function. With a framework, their code calls your function.

Phrasing correction Do not say "with a framework I don't have control". That sounds like the framework restricts you, and it invites a follow-up you don't want. Say **inverted** control. You keep full control of your business logic; what you hand over is the **timing and the flow** — when your code gets invoked.

	Library	Framework
Who calls whom	Your code calls it	It calls your code
Owns the flow	You do	It does
You provide	The call site	The fill-in-the-blanks pieces
Swappable?	Easily — it's one function call	Hard — your app is shaped around it
Examples	Axios, Lodash, Day.js, React*	Spring Boot, NestJS, Angular, Express*

* The two starred entries are the ones with an asterisk in real life — covered below.

2.2 What is a JavaScript runtime environment?

Say exactly this "A JavaScript engine only understands the language — variables, functions, objects. It has no idea what a file or a network socket is. A runtime environment is the engine **plus** the extra capabilities it needs to actually do useful work in a particular place. The browser is one runtime; Node.js is another."

Concretely, a runtime is three things bolted together:

- The engine** — V8 in both Chrome and Node. Parses and executes JavaScript. Holds the call stack and the heap.
- The host capabilities** — things JavaScript itself does not define. The browser adds `document`, `fetch`, `localStorage`. Node adds `fs`, `http`, `process`, `Buffer`.
- The event loop** — the scheduler that decides which callback runs next.

The line that lands "That's why `document.getElementById` fails in Node and `require('fs')` fails in a browser. Same language, same engine, different runtime — different capabilities around it."

2.3 What is Node.js?

Say exactly this — the one-sentence version "Node.js is a JavaScript runtime built on Chrome's V8 engine that lets you run JavaScript outside the browser, on the server. It's single-threaded with a non-blocking, event-driven I/O model, which is what makes it good at handling thousands of concurrent connections."

If they want thirty more seconds, add this and stop:

"Under V8 it uses a C library called libuv, which provides the event loop and the async I/O. Because I/O doesn't block the thread, one process can hold a very large number of open connections — so it suits APIs, real-time services and I/O-heavy workloads. The trade-off is that CPU-heavy work blocks that single thread, so you push that to worker threads or a separate service."

The trap in this question "Node.js is a framework" — wrong, and a common reflex answer. It is a **runtime**. Similarly, do not say "Node.js is a language". If you say nothing else correctly, say the word **runtime**.

2.4 What is Express.js?

Say exactly this "Express is a minimal, unopinionated web framework for Node.js. Node's built-in `http` module gives you a raw request and response — no routing, no body parsing, nothing. Express adds routing and a middleware pipeline on top of that, so you write `app.get('/orders', handler)` instead of parsing URLs by hand."

The follow-up that always comes: "what is middleware?"

Say exactly this "Middleware is a function that receives the request, the response and a `next` function. It can inspect or modify the request, then either end the response or call `next()` to pass control down the chain. Authentication, logging, body parsing and error handling are all just middleware. It's a pipeline — the request flows through in the order the middleware was registered."

"Minimal and unopinionated" means: it gives you routing and middleware and nothing else. No ORM, no dependency injection, no folder structure, no validation. You assemble those yourself. That is the contrast with NestJS and with Spring Boot.

2.5 What is React.js?

Say exactly this "React is a JavaScript library for building user interfaces, from Meta. You describe the UI as a tree of components, each component renders based on its state and props, and React updates the actual DOM for you when that state changes. It's declarative — you describe what the UI should look like for a given state, not the steps to mutate it."

Why React is a "library" and not a "framework" React handles the **view layer only**. It does not give you routing, data fetching, or global state — you add React Router, TanStack Query, Redux yourself. That is exactly why it is called a library. Compare Angular, which ships routing, HTTP, DI and forms in the box — that is a framework.

Fair caveat if pressed: React *does* call your component functions, which is framework-like inversion of control. The honest answer is: "Meta calls it a library because it's view-only and you compose the rest yourself — though the rendering model does invert control, so the line is a bit blurry in practice." That answer scores higher than a confident wrong one.

Two terms they may probe:

- **Virtual DOM** — "React keeps a lightweight in-memory representation of the UI. On a state change it builds a new one, diffs it against the previous one, and applies only the minimal set of real DOM changes. Direct DOM manipulation is slow, so batching the diff is the win."
- **Component** — "A reusable function that takes props and returns what should be rendered."

2.6 The pressure-recall drill

How to actually fix the retrieval problem Knowing an answer and being able to **produce it in one second while tired** are two different skills. The second one only comes from spoken repetition. Reading this page will not do it.

The protocol — ten minutes, three times before any interview

1. Cover the answers. Read only the prompt.
2. **Say the answer out loud.** Not in your head — the failure happened out loud, so the practice has to be out loud.
3. If you hesitate for more than two seconds, mark it and repeat that one three times.
4. Do the whole list in a random order the third time through.

Prompt	First eight words of the answer (the rest follows)
What is Node.js?	"A JavaScript runtime built on Chrome's V8 engine..."
What is Express?	"A minimal, unopinionated web framework for Node..."
What is React?	"A JavaScript library for building user interfaces..."
Library vs framework?	"A library is code I call, a framework calls me..."
What is a runtime?	"The engine plus the capabilities it needs..."
What is middleware?	"A function that gets request, response and next..."
What is npm?	"Node's package manager and its public registry..."
What is REST?	"An architectural style: resources as URLs, HTTP verbs..."
What is a Promise?	"An object representing a value not available yet..."
What is TypeScript?	"JavaScript with static types, compiled to plain JavaScript..."
What is MongoDB?	"A document database storing BSON documents in collections..."
What is Redis?	"An in-memory key-value store, used for caching..."
What is Docker?	"Packages an app with its dependencies into an image..."
What is an API?	"A contract that lets one system call another..."
What is Git?	"A distributed version control system for tracking changes..."

The recovery line — for when it happens anyway It will happen again at some point. Have a scripted recovery so a blank does not become a spiral:

"Give me one second, I've been deep in the internals all week and I want to give you the clean definition rather than ramble."

Then take the second and answer. Interviewers do not penalise a composed pause. They penalise panic, and they penalise a candidate who starts hedging and never lands on an answer.

3. CI/CD, Explained Through GitHub Actions

Why they split the question in two "Have you worked on CI/CD?" tests the concept. "Now explain it using GitHub Actions" tests whether you have actually opened the YAML file or only heard about the pipeline in standups. Answer the second half with **vocabulary from the file** — workflow, job, step, runner, trigger. That is the tell.

3.1 The concept, in two sentences

Say exactly this "Continuous Integration means every push automatically builds the code and runs the tests, so a broken change is caught in minutes rather than at release time. Continuous Delivery or Deployment takes the build that passed and ships it — to staging automatically, and to production either automatically or behind a manual approval."

The distinction they may probe: Continuous *Delivery* means every passing build is *ready* to ship, with a human pressing the button. Continuous *Deployment* removes the human. Most teams do delivery to production and deployment to staging.

3.2 The GitHub Actions vocabulary

Term	What it is
Workflow	A YAML file in <code>.github/workflows/</code> . One file, one pipeline.
Trigger (on)	What starts it — <code>push</code> , <code>pull_request</code> , <code>schedule</code> (cron), <code>workflow_dispatch</code> (manual button).
Job	A group of steps that share one machine. Jobs run in parallel by default; <code>needs:</code> makes one wait for another.
Runner	The machine a job runs on. <code>runs-on: ubuntu-latest</code> is GitHub-hosted; self-hosted runners are for private-network access.
Step	One command (<code>run:</code>) or one reusable action (<code>uses:</code>).
Action	A packaged reusable step, e.g. <code>actions/checkout@v4</code> , <code>actions/setup-node@v4</code> .
Secret	Encrypted value from repo or org settings, read as <code>\${{ secrets.NAME }}</code> . Masked in logs.
Artifact	A file passed between jobs or downloaded later — build output, coverage report.
Matrix	Runs the same job across several versions, e.g. Node 18 / 20 / 22, in parallel.
Environment	A named deploy target (<code>staging</code> , <code>production</code>) that can carry required reviewers — this is how you add manual approval.

3.3 A workflow you can describe from memory

```
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    # JOB 1 – runs on every push and PR
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage
    # caches ~/.npm between runs
    # ci, not install – respects the lockfile

  build:
    # JOB 2 – only if tests passed
    needs: test
    if: github.ref == 'refs/heads/main' # PRs stop after the test job
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and push image
        run: |
          echo "${{ secrets.REGISTRY_TOKEN }}" | docker login -u x --password-stdin
          docker build -t myapp:${{ github.sha }} .
          docker push myapp:${{ github.sha }}

  deploy:
    # JOB 3 – approval gate lives here
    needs: build
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: kubectl set image deploy/myapp myapp=myapp:${{ github.sha }}
```

The four details that make this sound lived-in

1. `npm ci` not `npm install` — `ci` deletes `node_modules` and installs exactly what the lockfile says, so builds are reproducible. `install` can quietly drift.
2. Tag the image with `github.sha`, never `latest` — the commit hash makes every deploy traceable and makes rollback a one-line image change.
3. `needs:` creates the dependency chain. Without it all three jobs fire simultaneously and you deploy code whose tests are still running.

4. Manual approval is not a step — it is the `environment:` key plus required reviewers configured on the repo.

3.4 The follow-ups

"How do you handle secrets?"

"Repository or organisation secrets, referenced as `${{ secrets.X }}`. GitHub masks them in logs. For cloud credentials the better pattern is OIDC — the workflow assumes an IAM role using a short-lived token, so there is no long-lived key stored anywhere."

"How do you speed up a slow pipeline?"

"Cache dependencies with `cache: 'npm'`. Split slow independent jobs to run in parallel instead of chaining them with `needs`. Only run the expensive jobs on `main`, using a path filter or an `if` condition. And use a matrix so version coverage runs concurrently rather than sequentially."

"What if a deploy breaks production?"

"Because the image is tagged with the commit SHA, rollback is redeploying the previous SHA — no rebuild needed. Beyond that, a health check on the new pods with an automatic rollback if they fail readiness, and for riskier changes a canary that shifts a small percentage of traffic first."

"What did YOU actually own?"

Prepare this — it is the question behind the question Have one concrete sentence ready about your own pipeline at Typo: what triggers it, what the stages are, where the tests run, how it reaches AWS or GCP, and one thing you changed or improved. Even a small one — "I added the lint stage", "I moved us to `npm ci` after a lockfile drift broke a build", "I added the staging approval gate". **A specific small contribution beats a general description of the whole pipeline.**

4. Jest — The Functions To Be Able To Name

Why this question exists It takes four seconds to separate someone who has written tests from someone who has watched tests run in CI. There is no depth here — only recall. Learn the table.

4.1 The core API

Function	What it does
<code>describe()</code>	Groups related tests into a block. Can nest.
<code>test()</code> / <code>it()</code>	Defines a single test case. The two are aliases.
<code>expect()</code>	Wraps the value under test, returning an object with matchers.
<code>beforeEach()</code> / <code>afterEach()</code>	Runs before / after every test in the block. Setup and teardown.
<code>beforeAll()</code> / <code>afterAll()</code>	Runs once for the whole block — DB connection, test server.
<code>jest.fn()</code>	Creates a mock function that records how it was called.
<code>jest.mock()</code>	Replaces an entire module with a mock.
<code>jest.spyOn()</code>	Wraps a real method to observe it, with the option to restore it.
<code>test.each()</code>	Runs the same test over a table of inputs.
<code>test.skip()</code> / <code>test.only()</code>	Skip a test, or run only that one.

4.2 The matchers

Matcher	Use
<code>toBe(x)</code>	Strict equality (<code>Object.is</code>). For primitives.
<code>toEqual(x)</code>	Deep equality. For objects and arrays. Knowing this pair is the real test.
<code>toStrictEqual(x)</code>	Deep equality that also checks <code>undefined</code> keys and class types.
<code>toBeNull()</code> , <code>toBeUndefined()</code> , <code>toBeTruthy()</code> , <code>toBeFalsy()</code>	Presence and truthiness checks.
<code>toContain(x)</code>	Array or string membership.
<code>toThrow()</code>	Asserts a function throws. Pass the call as an arrow function.
<code>toHaveBeenCalled()</code> / <code>toHaveBeenCalled(a)</code> / <code>toHaveBeenCalledTimes(n)</code>	Assertions on mocks.
<code>resolves</code> / <code>rejects</code>	Promise assertions: <code>await expect(p).resolves.toBe(1)</code> .
<code>not</code>	Negates any matcher: <code>expect(x).not.toBe(1)</code> .

```
describe('OrderService', () => {
  let repo;
  beforeEach(() => { repo = { save: jest.fn() }; });

  it('saves the order and returns its id', async () => {
    repo.save.mockResolvedValue({ id: 'ORD-1' });
    const result = await createOrder(repo, { total: 2499 });

    expect(repo.save).toHaveBeenCalledTimes(1);
    expect(result).toEqual({ id: 'ORD-1' });      // toEqual, not toBe
  });

  it('rejects a zero-total order', async () => {
    await expect(createOrder(repo, { total: 0 })).rejects.toThrow('Invalid total');
  });
});
```

The answer that scores highest Do not just name a function — name the pair and the distinction: " **toBe is strict equality, so it fails on two structurally identical objects because they're different references. toEqual does a deep comparison, which is what you want for objects and arrays. Mixing those two up is the most common Jest mistake I see.**" That single sentence demonstrates hands-on use more than listing ten function names.

Bridge from your Java background If Jest specifics feel thin, connect it: "The structure maps closely onto JUnit — `describe` is the test class, `it` is `@Test` , `beforeEach` is `@BeforeEach` , and `jest.fn()` plays the role Mockito's `mock()` does." Showing the mapping is honest and demonstrates the concept transfers.

5. The MongoDB Aggregation Question

This one you got right Your answer was correct and would have passed. This section is about the follow-ups that turn a correct answer into a strong one.

What you wrote

```
db.collection.aggregate([
  { $group: { _id: "$department", avgSalary: { $avg: "$salary" } } },
  { $project: { _id: 0, department: "$_id", avgSalary: 1 } }
])
```

Correct. `$group` buckets by department and `$avg` averages the salary within each bucket. The `$project` renames `_id` to `department` and suppresses the raw `_id`. Mixing `_id: 0` with inclusions is legal — `_id` is the one field allowed to be excluded alongside inclusions.

The polish

```
db.employees.aggregate([
  { $match: { status: "active" } },                                // 1. filter FIRST
  { $group: { _id: "$department",
    avgSalary: { $avg: "$salary" },
    headcount: { $sum: 1 } } },                                  // 2. context
  { $project: { _id: 0, department: "$_id",
    avgSalary: { $round: ["$avgSalary", 2] },                    // 3. readable
    headcount: 1 } },
  { $sort: { avgSalary: -1 } }                                    // 4. deterministic
])
```

The one principle they are listening for " \$match as early as possible." A `$match` at the front of the pipeline can use an index and shrinks the document set before the expensive stage runs. A `$match` after a `$group` cannot use an index at all, because it is filtering computed results that exist only in memory. Same for `$sort` — early it can use an index, late it is a blocking in-memory sort with a 100 MB per-stage limit.

The follow-ups worth having ready

Question	Answer
Only departments averaging above ₹10,00,000?	Add { \$match: { avgSalary: { \$gt: 1000000 } } } after \$group . That is the SQL <code>HAVING</code> equivalent — and say that word.
Add the department's name from another collection?	<code>\$lookup</code> with <code>from</code> , <code>localField</code> , <code>foreignField</code> , <code>as</code> . It produces an array, so follow it with <code>\$unwind</code> for a single match.
Highest-paid employee per department too?	<code>\$max: "\$salary"</code> for the number; <code>\$top</code> or a <code>\$sort</code> + <code>\$first</code> accumulator for the whole document.
What if the collection has 50 million documents?	Index the <code>\$match</code> field; add <code>allowDiskUse: true</code> for large groupings; and if it is a dashboard query, precompute with <code>\$merge</code> into a materialised view refreshed on a schedule.
How do you know it is fast?	<code>.explain("executionStats")</code> — check that stage one is an <code>IXSCAN</code> , not a <code>COLLSCAN</code> , and compare <code>nReturned</code> against <code>totalDocsExamined</code> .

6. "Implement an LRU Cache Instead of Redis"

The two halves of this question Half one is a classic data-structure implementation. Half two — the **"instead of Redis"** part — is an architecture question in disguise. Answer both. Most candidates answer only the first.

6.1 What LRU means

Least Recently Used. A cache with a fixed capacity. When it is full and a new item arrives, the item that has not been touched for the longest time is evicted. Both a read and a write count as "touching" it.

The requirement that shapes the design: `get` and `put` must both be $O(1)$. That rules out scanning to find the oldest entry.

6.2 The JavaScript answer — use Map

The insight that makes this short JavaScript's `Map` preserves insertion order. So the first key returned by `map.keys()` is always the oldest inserted. Deleting a key and re-setting it moves it to the back. That gives you the recency ordering for free — no linked list to hand-write.

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.map = new Map();
  }

  get(key) {
    if (!this.map.has(key)) return -1;
    const value = this.map.get(key);
    this.map.delete(key); // remove from its position...
    this.map.set(key, value); // ...and re-insert at the back = most recent
    return value;
  }

  put(key, value) {
    if (this.map.has(key)) {
      this.map.delete(key); // refresh position
    } else if (this.map.size >= this.capacity) {
      const oldest = this.map.keys().next().value; // first key = LRU
      this.map.delete(oldest); // evict
    }
    this.map.set(key, value);
  }
}
```

Verified: capacity 2, `put(1,1)`, `put(2,2)`, `get(1)` → 1, `put(3,3)` evicts 2, `get(2)` → -1, `put(4,4)` evicts 1, `get(1)` → -1, `get(3)` → 3, `get(4)` → 4.

$O(1)$ for both operations — `Map` `get`, `set` and `delete` are all $O(1)$, and `keys().next()` pulls the first entry without iterating the rest.

Three traps in this implementation 1. In `get`, forgetting the delete-then-set. Without it you have a plain capped map with FIFO eviction, not LRU — and it passes small tests, so it is easy to miss.

2. In `put`, evicting when the key *already exists*. If the key is present you are updating, not growing, so no eviction is needed. Hence the `else if`, not two separate `if`s.

3. Using `>` instead of `>=` in the size check — that lets the cache reach capacity + 1.

6.3 The classic answer — HashMap + doubly linked list

Be able to **describe** this even if you write the `Map` version, because it is what the interviewer has in mind and it is the language-independent answer.

- A **hash map** from key → node, giving $O(1)$ lookup.
- A **doubly linked list** holding the nodes in recency order: most recent at the head, least recent at the tail.
- **get**: look the node up in the map, unlink it from its current position, move it to the head.
- **put**: insert at the head; if over capacity, remove the tail node and delete its key from the map.

Why doubly and not singly linked: to unlink a node in $O(1)$ you need its predecessor. A singly linked list would require scanning from the head to find it, making the operation $O(n)$ and defeating the whole design.

The line to say "In JavaScript I'd use a `Map`, because it maintains insertion order — that gives me the recency list for free. The language-agnostic version is a hash map plus a doubly linked list: the map for $O(1)$ lookup, the list for $O(1)$ reordering. It has to be doubly linked so I can unlink a node without scanning for its predecessor."

6.4 The half most candidates skip: "instead of Redis"

An in-process LRU and Redis are not competitors — they sit at different layers. Name the trade-off explicitly.

	In-process LRU	Redis
Latency	Nanoseconds — a memory read	Sub-millisecond, but a network hop
Shared across pods?	No. Each pod has its own copy	Yes — one source of truth
Survives restart?	No — cold on every deploy	Yes, with persistence
Invalidation	Hard — you must reach every pod	Easy — delete one key
Cost	Free — uses app heap	Separate service to run and pay for
Memory pressure	Competes with your app's heap	Isolated

The senior answer "They solve different problems, so I'd usually run both. An in-process LRU as L1 for very hot, small, slow-changing data — feature flags, config, tenant metadata — where a network hop per request is pure waste. Redis as L2 for anything that has to be consistent across pods or survive a restart. The thing to be careful about with in-process caching is **invalidation**: with ten pods you have ten copies, so a stale entry can live in one pod after the data changed. That's why I keep TTLs short on the in-process layer and put anything correctness-critical in Redis."

Mention the library "In production I wouldn't hand-roll it — I'd use `lru-cache` from npm, which adds TTL, max size by weight, and stale-while-revalidate. Writing it by hand is the interview exercise; using the battle-tested one is the job." Saying this shows judgement without dodging the coding task.

7. The Project Questions

7.1 "The most challenging project you have worked on"

What is actually being assessed Not the project. **Your judgement.** They want to hear a real constraint, a decision you made under it, a trade-off you consciously accepted, and a number. A tour of the architecture answers a question nobody asked.

The five-beat structure — roughly two minutes

Beat	Length	Content
1. Context	2 sentences	What the product is, who uses it, your role. No more.
2. The challenge	2-3 sentences	The specific thing that was hard. Be concrete about <i>why</i> it was hard.
3. What you did	The bulk	The design, and at least one decision with an alternative you rejected and why.
4. The result	1-2 sentences	A number. Always a number.
5. What you'd change	1 sentence	One honest limitation. This beat is what separates senior from mid.

The most common failure — avoid this Spending ninety seconds on beat 1. The interviewer does not need the org chart or the product history. Two sentences, then move. If you find yourself explaining what Typo does in detail, you have already lost the question.

Applied to "Ask AI"

Beat 2 — make the challenge specific. "It was a RAG pipeline" is not a challenge. Pick the genuinely hard constraint. Strong candidates from that system:

- **Per-tenant data isolation** — retrieval must never leak one customer's data into another customer's answer, and that has to hold at the vector-search layer, not just at the API boundary.
- **Grounding and hallucination control** — the answer must be traceable to retrieved context, and the system must be able to say it does not know.
- **Retrieval quality** — chunking strategy, embedding choice, and why naive top-k similarity was not enough.
- **Tool-calling over internal APIs** — deciding when to retrieve versus when to call a live API for current data.

Beat 3 — the trade-off sentence to include "We chose RAG over fine-tuning deliberately. Fine-tuning would have to be redone every time the underlying data changed, the cost scales with retraining rather than with queries, and with enterprise customers we needed hard data isolation per tenant — which you cannot get from a single fine-tuned model that has seen everyone's data. RAG keeps the data outside the model, so freshness and isolation both come from the retrieval layer."

Beat 5 — a limitation to name honestly Pick one and have it ready. For example: retrieval quality on very short or ambiguous queries; the evaluation gap — measuring answer quality systematically rather than by spot-checking; or latency from the embedding plus retrieval plus generation chain. Naming a real limitation reads as maturity. Claiming there were none reads as inexperience.

7.2 "Which LLM model — name it exactly"

Why this question is asked, and why vagueness is costly It is a **hands-on check.** Anyone can say "we used an LLM". Only someone who configured the client knows the model string, the context window, and why that model over the alternative. A vague answer here retroactively casts doubt on the whole project story — the interviewer starts wondering whether you built it or described someone else's work.

Do this before the next interview — a fifteen-minute task

Open the code and write down the exact facts. Do not reconstruct them from memory in the room.

1. **The generation model string**, exactly as it appears in the config — the full versioned identifier, not the family name.
2. **The provider** — direct API, or via a cloud gateway such as Bedrock or Vertex.
3. **The embedding model**, separately. It is almost always a different model from the generation one, and forgetting that it is a separate choice is a common tell.
4. **The vector store** — and whether it is a dedicated one or an index inside your existing database.
5. **Your chunk size and overlap**, and the **top-k** you retrieve.
6. **Whether you evaluated alternatives**, and on what basis — cost per token, latency, context window, quality on your own eval set.

The shape of the answer — fill in your own values "For generation we're on **[exact model string]** via **[provider]**. For embeddings we use **[embedding model]**, stored in **[vector store]**. We chunk at roughly **[N]** tokens with **[M]** overlap and retrieve top **[k]**. We picked that generation model over **[alternative]** mainly on **[cost / latency / context window / quality on our eval set]** — and the model choice is config-driven, so swapping it is a deployment change rather than a code change."

If you genuinely cannot recall it in the room Do not guess a model name. A wrong version string is worse than an admission. Say: **"The model choice is config-driven and I'd have to check the exact version string rather than guess at it — but I can tell you why we chose that family: [reason]. The abstraction was deliberate, so switching models is a config change."** Then pivot to the architecture, which you do know cold. Honest plus architectural depth recovers most of the marks. A fabricated version string that the interviewer recognises as wrong loses all of them.

7.3 The pattern across both project questions

One rule **Every project claim needs one number attached.** How many tenants. How many documents indexed. What the p95 latency is. What percentage of support queries it deflected. Which number does not matter nearly as much as having one — a number is the difference between a story and a claim, and it is the single cheapest upgrade to these answers.

Appendix A — Self-Test

Cover the document. Answer out loud. Anything that takes more than two seconds goes back on the drill list.

Definitions (Section 2)

1. What is Node.js? Which single word must be in the answer?
2. What is Express? What do "minimal" and "unopinionated" mean concretely?
3. What is React? Why is it called a library rather than a framework?
4. Library vs framework — give the one-line rule and the term for it.
5. What is a JavaScript runtime? Name its three parts.
6. Why does `require('fs')` fail in a browser?
7. What is middleware? What are its three parameters?
8. What is the Virtual DOM and what problem does it solve?

Meeting rooms (Section 1)

9. Why can counting overlaps pairwise never find the right answer?
10. State the tie rule. Which test case exposes it?
11. Write the two-array solution from memory. Which operator handles the tie rule?
12. Why does Approach B satisfy the no-mutation requirement without a defensive copy?
13. What does a sort comparator returning `NaN` do, and why is that dangerous?
14. Why is a min-heap a poor first choice in JavaScript specifically?
15. Give the correct overlap condition for two intervals when touching endpoints do not conflict.

CI/CD and Jest (Sections 3-4)

16. Define workflow, job, step, runner, action.
17. Why `npm ci` rather than `npm install` in a pipeline?
18. Why tag images with the commit SHA rather than `latest` ?
19. How do you add a manual approval gate in GitHub Actions?
20. Difference between continuous delivery and continuous deployment.
21. `toBe` vs `toEqual` — when does each fail?
22. Name three Jest lifecycle hooks and when each runs.
23. How do you assert that an async function rejects?

MongoDB and caching (Sections 5-6)

24. Why must `$match` come as early as possible?
25. What is the aggregation equivalent of SQL's `HAVING` ?
26. Why does `$lookup` usually need `$unwind` after it?
27. Write the `Map`-based LRU cache from memory. What does `get` do besides return the value?
28. Why must the classic LRU list be doubly linked?
29. Name three things Redis gives you that an in-process LRU cannot.
30. What is the main risk of in-process caching across ten pods?

Appendix B — The Action Plan

Before the next interview — non-negotiable

#	Task	Time	Why
1	Memorise the fifteen definitions in the Section 2.6 drill. Out loud, three passes, randomised order.	30 min	This is what actually lost the round
2	Open the "Ask AI" config and write down the six facts in Section 7.2 on one card	15 min	Removes the vaguest answer you gave
3	Re-solve minMeetingRooms from a blank file. Approach B. No looking.	20 min	Cement the sweep pattern
4	Write the LRU cache from a blank file and test it	15 min	Very likely to recur — it is a standard screen
5	Write your own two-minute "most challenging project" script and say it aloud twice	30 min	Spoken rehearsal, not mental
6	Find your team's GitHub Actions file. Read it. Note one thing you changed in it.	15 min	Turns a general answer into a specific one

The habit changes

1. **Start every prep session with ten minutes of basics, not internals.** The deep material is more interesting, which is exactly why it crowds out the cheap material that gets asked first.
2. **Rehearse out loud.** The failure was a speaking failure. Silent reading does not train the same pathway.
3. **Before every live-coding round, run the checklist:** restate the edge-case rule → copy the input → brute force first → dry-run a hard case → re-read the comparator.
4. **Carry one number per project claim.** Write them on a card and keep it visible during remote rounds.
5. **Keep the depth.** Nothing in this document says the internals work was wasted. It wins the rounds that come after this one. It simply is not what an L1 screen measures.

The **honest summary** You prepared for a hard interview and got an easy one, and the easy one is harder to prepare for precisely because none of it feels worth rehearsing. The material in Sections 2, 4 and 6 takes under two hours to lock in and covers most of what went wrong. The engineering knowledge was never the problem — the retrieval was.